A State-Space Framework for Real-Time Win Probability Estimation in Twenty20 Cricket

Prateek Sharma*, Hanamanthagouda Policepatil†, Amit Arora†

Abstract

We develop a state-space framework for Twenty20 cricket that models match evolution through three latent dynamics: batting strength, scoring momentum, and match pressure. These states are updated ball-by-ball using Kalman filtering and mapped to win probabilities through a supervised calibration layer. The framework is evaluated using ball-by-ball data from 1,161 Indian Premier League matches spanning 18 seasons (2008–2025) and benchmarked against logistic regression, random forest, and gradient boosting models. The proposed approach delivers competitive predictive performance, consistently outperforming a static linear benchmark and performing particularly well in one-sided matches. In addition to forecasting, the framework provides an interpretable representation of match evolution through latent state trajectories. Momentum emerges as the earliest indicator of chase success, while pressure provides the most persistent distinction between successful and unsuccessful chases. Batting strength acts primarily as a late-innings confirmation signal. The results demonstrate that state-space modelling can combine accurate forecasting with interpretable and temporally coherent representations of match dynamics for real-time cricket analytics.

Keywords: T20 cricket, Indian Premier League, Win probability, State-space models, Kalman filtering, Latent variable models

*Corresponding Author. University of South Carolina, SC, USA.
†Indian Institute of Technology Gandhinagar, India
†Indian Institute of Technology Gandhinagar, India

1 Introduction

The increasing commercialisation and global viewership of Twenty20 cricket has amplified demand for real-time quantitative assessments of match state for broadcasters, analysts, team strategists, betting markets, and fans. Among these, win probability, the estimated likelihood that a team will ultimately win given the current match situation has become one of the most widely used summary measures in sports analytics, with established applications in Baseball (Tango et al., 2007), American football (Lock and Nettleton, 2014), and Basketball (Stern, 2005). More recently, sports analytics research has expanded beyond sport-specific prediction models toward general statistical frameworks for estimating win probability, expected value, and team strength across a wide range of sporting contexts (Baumer et al., 2023; Albert, 2015).

In cricket, recent studies have shown that observable match-state variables such as runs scored, wickets in hand, overs remaining, and run-rate differentials contain substantial predictive information, enabling logistic regression and tree-based ensemble methods to produce accurate in-play win probability estimates (Asif and McHale, 2016; Pathak and Wadhwa, 2016; Wickramasinghe, 2022). Beyond traditional regression-based approaches, recent studies have explored probabilistic models, ensemble learning techniques, and deep learning architectures for forecasting cricket outcomes, reflecting the increasing methodological diversity within cricket analytics (Senevirathne and Manage, 2021; Goel et al., 2021). Similar predictive analytics frameworks have recently been developed for other sports, demonstrating the growing use of machine learning techniques for real-time match outcome forecasting across sporting domains (Wong et al., 2025)

While these approaches have demonstrated strong predictive performance, they are primarily designed to map the current observable match state directly to an outcome probability. As a result, they provide limited insight into the underlying dynamics through which a match evolves. Similar limitations have motivated the development of dynamic statistical models in other sports, where latent states are used to represent evolving team strength and match dynamics rather than relying solely on contemporaneous observations(Glickman and Stern, 2005; Koopman and Lit, 2015). In practice, analysts,

commentators, and fans frequently describe matches in terms of concepts such as batting strength, scoring momentum, and match pressure, latent constructs that are not directly observed but are inferred from sequences of events over time. Explicitly modelling these dynamics offers a complementary perspective to existing prediction frameworks, providing both win probability estimates and an interpretable representation of how a match develops.

T20 cricket is inherently dynamic. Match conditions evolve continuously across the powerplay, middle overs, and death overs, with each phase characterised by different scoring patterns, tactical objectives, and levels of risk. Consequently, identical scorecards can arise from very different underlying trajectories of play. This temporal dependence suggests that models treating observations as independent snapshots may overlook important information contained in the evolution of the innings itself.

To capture these dynamics, we develop a state-space framework in which *Batting Strength*, Scoring *Momentum*, and Match *Pressure* evolve as latent states and are updated ball-by-ball using Kalman filtering. Information from recent scoring patterns and match context is incorporated sequentially after every delivery, allowing the model to track how a chase develops over time(Durbin and Koopman, 2012). State-space models have been widely adopted for sequential forecasting because they provide a principled framework for separating latent system evolution from noisy observations while recursively updating beliefs as new information becomes available (Harvey and Harvey, 1990; Simon, 2006). The resulting latent-state estimates are translated into win probabilities through a supervised calibration approach. This architecture combines real-time forecasting with an interpretable representation of match evolution, linking win probability estimates to the underlying dynamics that drive them.

The framework is evaluated using ball-by-ball data from 1,161 Indian Premier League (IPL) matches spanning 18 seasons (2008–2025). Performance is benchmarked against logistic regression, random forest, and gradient boosting using temporally separated training, validation, and test sets. The results show that the proposed state-space approach consistently outperforms a static linear benchmark and remains competitive with leading

machine-learning methods while providing substantially greater interpretability. Analysis of the estimated latent states reveals distinct patterns of match evolution: *Momentum* differentiates successful and unsuccessful chases early in the innings, *Pressure* provides the most persistent separation between outcomes, and *Batting Strength* emerges primarily as a late-innings signal. Together, these latent trajectories provide an interpretable representation of how and why win probability changes over time, complementing existing prediction-focused approaches to cricket analytics.

The remainder of this paper is organised as follows. Section 2 presents the state-space modelling framework. Section 3 describes the data, feature engineering, and preprocessing pipeline. Section 4 details model specification, parameter estimation, and benchmark configurations. Section 5 reports empirical results. Section 6 discusses the implications of the findings. Section 7 concludes with limitations and directions for future research.

2 Methodology

2.1 Modeling Objective and Framework

We develop a real-time win probability model for Twenty20 cricket within a state-space framework estimated using Kalman filtering (Kalman, 1960; Harvey and Harvey, 1990; Simon, 2006), using the Indian Premier League (IPL) as the empirical setting for model development and evaluation. Each match is treated as a dynamic stochastic system observed at ball-level resolution. Latent match dynamics evolve over time and are updated sequentially using scoring information and contextual match variables. The resulting state estimates are mapped to calibrated win probabilities after every delivery.

Although individual ball outcomes are discrete, the observation vector consists entirely of rolling aggregates over 6–24 ball windows and the current run rate. These quantities are substantially smoother than the underlying scoring events and are reasonably approximated by Gaussian distributions, making the linear Gaussian framework a practical working approximation (Glickman and Stern, 2005). More flexible nonlinear or non-Gaussian state-space models are discussed in Section 7.

The latent states are estimated using deliveries from both innings, whereas the probability calibration stage is trained exclusively on second-innings deliveries, where a binary win/loss outcome is well defined at ball level. Each delivery is indexed by discrete time step $t = 1, 2, \ldots, T$, where T denotes the total number of deliveries in the match, including extras.

2.2 State-Space Representation

Let $x_t \in \mathbb{R}^3$ denote the latent match state at ball t, defined as:

$$x_t = \begin{bmatrix} S_t \\ M_t \\ P_t \end{bmatrix}$$

where S_t represents latent *Batting Strength*, M_t captures short-term scoring *Momentum*, and P_t measures match *Pressure*. Batting strength evolves gradually and reflects underlying scoring ability, momentum responds to recent fluctuations in scoring, and pressure summarizes the strategic difficulty of the current match situation. Positive values of S_t and M_t favour the batting side, whereas positive values of P_t indicate increasing pressure in favour of the bowling side.

These quantities are not directly observed and are inferred sequentially from the rolling-window observation vector y_t described in Section 2.3. The latent state evolves according to the linear Gaussian transition equation

$$x_t = Ax_{t-1} + Bu_t + w_t, \tag{1}$$

where $A \in \mathbb{R}^{3\times3}$ denotes the state transition matrix, $u_t \in \mathbb{R}^8$ is a vector of observable match-context variables, $B \in \mathbb{R}^{3\times8}$ maps these controls into state adjustments, and $w_t \sim \mathcal{N}(0, Q)$ represents process noise with covariance matrix $Q \in \mathbb{R}^{3\times3}$.

The control vector contains information available up to delivery t, including balls remaining, wickets in hand, run-rate differential, required run rate, a composite pressure index, and indicators for the powerplay, middle overs, and death overs. All variables

are constructed using only contemporaneous and historical information, thereby avoiding look-ahead bias. The specification and estimation of A, B, and Q are described in Section 4.

2.3 Observation Model

At each delivery t, the Kalman filter observes a vector of match statistics $y_t \in \mathbb{R}^{13}$ that provide noisy signals about the latent state. The observation equation is

$$y_t = Hx_t + v_t, \qquad v_t \sim \mathcal{N}(0, R), \tag{2}$$

where $H \in \mathbb{R}^{13\times 3}$ maps the latent states into observable quantities and $R \in \mathbb{R}^{13\times 13}$ denotes the observation noise covariance matrix. The matrix H is estimated from the data using the Expectation–Maximization procedure described in Section 4. Estimating the observation matrix jointly with the remaining state-space parameters allows the latent representation to be learned directly from the observed data rather than imposed a priori (Ghahramani and Hinton, 1996).

The observation vector consists of 13 rolling-window features computed using information available prior to delivery t:

- Runs scored: runs accumulated over the previous 6, 12, and 24 balls.
- Wickets lost: wickets lost over the previous 6, 12, and 24 balls.
- Dot balls: dot-ball counts over the previous 6, 12, and 24 balls.
- Boundaries: boundary counts over the previous 6, 12, and 24 balls.
- Current run rate: the run rate at delivery t.

All features are constructed strictly from past information, thereby eliminating look-ahead bias. Variables are standardised to zero mean and unit variance before estimation. Because the observation vector is composed of rolling aggregates and rates rather than raw ball outcomes, the Gaussian observation assumption represents a reasonable working approximation.

2.4 Sequential Estimation via Kalman Filtering

The latent state is updated recursively after each delivery. Let $\hat{x}_{t|t}$ denote the filtered state estimate after observing ball t, and let $\Sigma_{t|t} \in \mathbb{R}^{3\times 3}$ denote its covariance matrix, which measures uncertainty in the estimate.

At each delivery, the Kalman filter alternates between a prediction step and an update step.

Prediction Step

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} + Bu_t \tag{3}$$

$$\Sigma_{t|t-1} = A\Sigma_{t-1|t-1}A^\top + Q \tag{4}$$

Before observing ball t, the model projects the latent state forward using the previous estimate and the current match context. The covariance update increases uncertainty through the process noise covariance matrix Q, allowing for random variation in match dynamics.

Update Step

$$K_t = \Sigma_{t|t-1}H^\top(H\Sigma_{t|t-1}H^\top + R)^{-1} \tag{5}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - H\hat{x}_{t|t-1}) \tag{6}$$

After observing ball t, the model updates its prediction using the innovation term $(y_t - H\hat{x}_{t|t-1})$, which measures the gap between predicted and observed match statistics. The Kalman gain K_t determines how strongly the new observation influences the updated state estimate. Larger state uncertainty increases the weight placed on new observations, while larger observation noise reduces it.

The covariance matrix is updated using the Joseph form:

$$\Sigma_{t|t} = (I - K_tH)\Sigma_{t|t-1}(I - K_tH)^\top + K_tRK_t^\top \tag{7}$$

The Joseph form (Bierman, 2006) is numerically more stable than the standard covariance update and guarantees that $\Sigma_{t|t}$ remains positive semi-definite under finite-precision computation (Grewal and Andrews, 2014).

2.5 Mapping Latent State to Win Probability

The Kalman filter produces filtered state estimates $(\hat{S}_t, \hat{M}_t, \hat{P}_t)$ representing batting strength, scoring momentum, and match pressure after each delivery. These states are mapped to win probabilities using a supervised logistic regression model estimated from second-innings deliveries.

Let W_t denote wickets in hand, $R_{24,t}$ the runs scored over the previous 24 deliveries, RRR_t the required run rate, ΔRR_t the run-rate differential, and B_t the number of balls remaining. Define the feature vector $z_t \in \mathbb{R}^{10}$ as

$$z_t = \begin{bmatrix} \hat{S}_t & \hat{M}_t & \hat{P}_t & \hat{S}_t W_t & \hat{M}_t R_{24,t} \\ \hat{P}_t \mathrm{RRR}_t & B_t & W_t & \Delta RR_t & \mathrm{RRR}_t \end{bmatrix}^\top \tag{8}$$

The first three elements correspond to the filtered latent states. The next three are interaction terms that allow the predictive effect of each state to depend on the current match context. The remaining variables summarize the observable state of the chase.

Win probability is estimated using an L2-regularised logistic regression model (Hosmer Jr et al., 2013; James et al., 2021).

$$\Pr(\mathrm{Win}_t = 1) = \frac{1}{1 + \exp(-\beta^\top z_t)}, \tag{9}$$

2.6 Temporal Weighting Function

The model is estimated using IPL data spanning 2008–2025. Because scoring patterns and strategic behaviour have evolved substantially over this period, treating all seasons equally would allow outdated match characteristics to influence parameter estimates disproportionately.

To emphasize contemporary T20 cricket, observations are assigned season-specific

weights during both the EM estimation stage and the logistic calibration stage. The weighting scheme is intended to mitigate the influence of concept drift arising from evolving scoring environments across seasons (Gama et al., 2014).We employ a piecewise linear weighting scheme with a lower bound of 0.5:

$$w(\text{year}) = \max\left(0.5,\ 1 - 0.05(2022 - \text{year})\right)$$

with seasons from 2022 onward assigned unit weight.

Under this specification, seasons from 2022 onward receive full weight, while earlier seasons are progressively downweighted. The weighting scheme reduces the influence of obsolete scoring environments while preserving information from earlier eras that remains relevant to match dynamics.

2.7 Initialization and Innings Conditioning

The Kalman filter requires initial values for both the latent state vector and its covariance matrix. We set

$$\Sigma_{0|0} = I_3,$$

which provides moderate prior uncertainty and allows the filter to adapt rapidly during the early stages of an innings.

Initialization differs between the two innings because batting first and chasing a target represent fundamentally different situations. Initial state specification is a standard component of state-space modelling because filtering performance during the early stages of a sequence depends on the choice of prior state estimates and covariance matrices (Durbin and Koopman, 2012).

First innings (setting a target):

$$S_0 = \frac{\text{venue}_{\text{avg}} - 150}{30}, M_0 = 0, P_0 = 0$$

Since no target or scoring trend exists at the start of the innings, momentum and pressure are initialized at zero. Batting strength depends on the historical average first-innings score at the venue, with a default value of $\text{venue}_{\text{avg}} = 175$ used when venue-specific information is unavailable.

Second innings (chasing a target):

$$\begin{aligned} S_0 &= -0.5\,\text{norm}_{\text{target}}, \\ M_0 &= 0, \\ P_0 &= \max\left(-2, \min\left(4, \text{norm}_{\text{target}}\right)\right). \end{aligned}$$

where

$$\text{norm}_{\text{target}} = \frac{\text{target} - \text{venue}_{\text{avg}}}{20}.$$

Momentum is again initialized at zero. Batting strength is lower for above-par targets, reflecting the disadvantage faced by the chasing side, while initial pressure increases with target difficulty. Bounds are imposed on P_0 to avoid extreme initial values in unusually high or low-scoring matches. The choice of initialization constants and scaling parameters is discussed in Appendix A.3.

3 Data and Feature Engineering

3.1 Data Source and Scope

The dataset comprises ball-by-ball records from the IPL obtained from Cricsheet,[1] covering 1,161 matches across the 2008–2025 seasons. Each record contains delivery outcomes, innings and over information, player identities, and match results. After preprocessing, the dataset contains 277,399 deliveries.

Matches decided using the Duckworth-Lewis-Stern (DLS) method, abandoned matches,

[1] `https://cricsheet.org`

and matches with incomplete ball-by-ball records were excluded. Super overs were retained in the raw data but omitted from model estimation and evaluation because their six-ball structure differs fundamentally from that of a standard T20 innings.

Extras (wides and no-balls) were retained as observed events, although the balls-remaining calculation was updated only after valid deliveries. Venue names were standardized across seasons to ensure consistent estimation of venue-specific scoring averages used during state initialization.

The state-space model is estimated using deliveries from both innings, whereas the supervised calibration stage is trained exclusively on second-innings deliveries, where a binary win/loss outcome is naturally defined at ball level.

The complete feature set consists of 23 observable variables derived from rolling scoring statistics and match-context information. These variables enter different components of the framework through separate observation, control, and calibration matrices, while the benchmark machine-learning models use the full feature set directly. The calibration layer additionally incorporates the estimated latent states and interaction terms. A detailed feature-to-matrix mapping is provided in Appendix A1.

To avoid information leakage, training, validation, and test splits were constructed chronologically at the season level. Details of the data partition are provided in Section 3.3.

3.2 Feature Engineering

Features are constructed at ball-level resolution and are strictly backward-looking to avoid look-ahead bias. Rolling-window aggregates are computed using a one-ball shift, ensuring that every feature at delivery t depends only on information available up to delivery $t-1$.

In total, 23 observable features are constructed and organized into four groups. These features are used differently across the state-space model, benchmark machine-learning models, and the win-probability calibration layer. A complete feature-to-matrix mapping is provided in Appendix Table A1.

3.2.1 Group A: Match-State Features

Instantaneous match-state variables describe the current game situation and include quantities such as current run rate, wickets in hand, balls remaining, required run rate, run-rate differential, pressure index, and innings number. These variables provide contextual information about the state of the innings.

3.2.2 Group B: Rolling-Window Features

Short-term match dynamics are captured through rolling aggregates computed over the previous 6, 12, and 24 deliveries. Four statistics are recorded at each window length: runs scored, wickets lost, dot balls, and boundaries, yielding 12 features in total. These rolling statistics constitute the primary information source for the Kalman filter observation vector.

Because the observation vector consists of rolling aggregates and rates rather than raw ball outcomes, the Gaussian observation assumption is substantially more reasonable than it would be for individual delivery-level events.

3.2.3 Group C: Phase Indicators

Three binary indicators represent the powerplay (overs 1–6), middle overs (7–15), and death overs (16–20). These variables capture structural differences in scoring patterns and tactical behaviour across innings phases.

3.2.4 Group D: Venue Features

Venue-specific information is incorporated through the historical average first-innings score and a par-deviation measure, which quantifies the extent to which the batting side is ahead of or behind the expected scoring trajectory.

3.2.5 Feature Scaling

All features are standardized using parameters estimated from the training set only. The same transformations are applied to the validation and test sets without refitting,

preventing information leakage from held-out data.

3.3 Data Splits

To preserve temporal ordering and avoid information leakage, all data partitions are defined at the season level. Seasons are never divided across splits, ensuring that models are always trained on historically prior data and evaluated on future seasons.

The dataset is partitioned into training (2008–2020), validation (2021–2022), and test (2023–2025) sets. The training set is used for parameter estimation and model fitting, the validation set for model selection and tuning, and the test set is reserved exclusively for final evaluation.

Split	**Seasons**	**Matches**	**Deliveries**
Training	2008–2020	812	193,245
Validation	2021–2022	134	32,337
Test	2023–2025	215	51,817
Total	2008–2025	**1,161**	**277,399**

Table 1: Season-level train, validation, and test splits.

Win-probability labels are defined only for second-innings deliveries, where match outcomes are naturally associated with each ball. First-innings deliveries contribute to state estimation and phase-level diagnostics but are excluded from supervised probability calibration and the aggregate predictive evaluation reported in Table 5.

3.4 Evolution of the IPL Scoring Environment

The final sample comprises 1,161 IPL matches and 277,399 deliveries spanning the 2008–2025 seasons. Over this period, scoring patterns evolved substantially. Average first-innings totals increased from approximately 161 runs in 2008 to more than 190 runs by 2025, accompanied by higher boundary frequencies and lower dot-ball rates, reflecting the increasingly aggressive batting environment of modern T20 cricket.

These patterns highlight the non-stationary nature of the game. Match situations that represented above-par performances in earlier seasons correspond to more modest

scores in the contemporary IPL. Consequently, treating all seasons equally risks allowing outdated scoring environments to dilute the representation of modern T20 cricket. This motivates the temporal weighting scheme described in Section 2, which assigns greater importance to recent seasons while retaining information from earlier eras.

4 Model Formulation

4.1 Kalman Filter Parameter Specification

The proposed framework is implemented as a linear Gaussian state-space model with three latent states and eight control inputs. The latent state evolves according to

$$x_t = Ax_{t-1} + Bu_t + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, Q),$$

where $x_t = (S_t, M_t, P_t)^\top$ denotes the latent state vector and $u_t \in \mathbb{R}^8$ is the control vector defined in Section 3.2. The transition matrix (A), control matrix (B), and process noise covariance matrix (Q) are estimated from the training data using the Expectation–Maximization procedure described in Section 2. Their estimated values are reported below.

4.1.1 Transition Matrix A

The transition matrix was initialized as

$$A_0 = \begin{bmatrix} 0.85 & 0 & 0 \\ 0 & 0.60 & 0 \\ 0 & 0 & 0.80 \end{bmatrix},$$

reflecting the prior belief that batting strength evolves gradually, momentum is more transient, and pressure exhibits intermediate persistence. These values serve only as starting points for the EM algorithm. Similar persistence-based initialization strategies are commonly adopted in state-space estimation because they improve numerical stability

while allowing the data to determine the final parameter values during iterative estimation (Shumway and Stoffer, 1982).

To improve numerical stability and preserve the domain-informed structure, EM updates were damped according to

$$A^{(k+1)} = 0.9A^{(k)} + 0.1A_{\text{EM}}^{(k+1)},$$

where $A_{\text{EM}}^{(k+1)}$ denotes the transition matrix obtained from the M-step. The final estimated transition matrix is

$$\hat{A} = \begin{bmatrix} 0.887 & 0.039 & -0.028 \\ -0.027 & 0.935 & -0.015 \\ 0.006 & -0.003 & 0.907 \end{bmatrix},$$

with rows and columns corresponding to batting strength (S_t), momentum (M_t), and pressure (P_t).

The estimated matrix is dominated by large diagonal elements, indicating substantial persistence in all three latent states. In particular, the persistence of momentum increases considerably relative to its initial value, suggesting that short-term scoring patterns are more sustained than assumed a priori. The off-diagonal terms are comparatively small, implying limited but non-negligible interactions among the latent states.

4.1.2 Control Matrix B

The control matrix $B \in \mathbb{R}^{3\times 8}$ maps the observable match-context variables in u_t to increments in the latent state vector. The matrix was initialized from $\mathcal{N}(0, 0.01)$ with no sign restrictions and estimated jointly with the remaining state-space parameters via the EM procedure using a damping rate of 0.40. The final estimated matrix is reported in Table 2.

Control Variable	S_t	M_t	P_t
Balls remaining	-0.0065	-0.0210	$+0.0448$
Wickets in hand	-0.0275	$+0.0121$	-0.0440
Run-rate differential	-0.0131	-0.0202	$+0.0096$
Pressure index	$+0.0046$	$+0.0091$	-0.0054
Powerplay	$+0.0019$	$+0.0102$	$+0.0106$
Middle overs	-0.0025	-0.0106	-0.0097
Death overs	$+0.0010$	$+0.0014$	-0.0002
Required run rate	$+0.0046$	$+0.0091$	-0.0054

Table 2: Estimated control matrix $\hat{B} \in \mathbb{R}^{3\times 8}$ after EM convergence. Entries represent the marginal effect of a one-standard-deviation change in each standardized control variable on the corresponding latent-state increment.

Several estimated coefficients align with cricketing intuition. Additional wickets in hand reduce latent pressure, reflecting the greater flexibility available to the batting side, while the powerplay phase increases momentum, consistent with the more aggressive scoring encouraged by fielding restrictions. Higher run-rate differentials are associated with lower momentum, suggesting that teams falling behind the required pace struggle to sustain scoring intensity.

Most coefficients are relatively small in magnitude, indicating that the control variables act primarily as incremental adjustments to the latent-state dynamics rather than dominating the state evolution.

4.1.3 Observation Matrix H

The observation matrix $H \in \mathbb{R}^{13\times 3}$ maps the latent states to the 13-dimensional observation vector y_t. The matrix was initialized with all entries equal to 0.1, imposing no prior structure on the relationship between latent states and observed variables. During the EM procedure, $\hat{H}$ was updated according to

$$H_{\text{new}} = \left(\sum_t y_t \hat{x}_{t|t}^\top\right)\left(\sum_t \hat{x}_{t|t}\hat{x}_{t|t}^\top + \lambda I\right)^{-1},$$

with damping rate $\alpha = 0.40$ and ridge regularization parameter $\lambda = 0.05$. Because $\hat{H}$ governs the relationship between observed match statistics and latent states, it is the parameter matrix most directly informed by the data. Joint estimation of the observa-

tion matrix and latent states is one of the principal advantages of EM-based state-space estimation, allowing measurement relationships to be learned directly from the observed data rather than specified a priori (Ghahramani and Hinton, 1996).The final estimated matrix is reported in Table 3.

Observation	S_t	M_t	P_t
Runs (6 balls)	0.086	1.257	-0.244
Runs (12 balls)	0.262	1.567	-0.180
Runs (24 balls)	0.444	1.198	0.010
Wickets (6 balls)	1.354	-0.574	0.808
Wickets (12 balls)	1.759	-0.817	1.167
Wickets (24 balls)	1.231	-0.548	0.942
Dot balls (6 balls)	-0.592	-0.280	0.976
Dot balls (12 balls)	-0.889	-0.114	1.281
Dot balls (24 balls)	-0.756	0.226	1.013
Boundaries (6 balls)	-0.124	1.155	-0.066
Boundaries (12 balls)	-0.045	1.523	0.008
Boundaries (24 balls)	0.104	1.246	0.114
Current run rate	0.318	0.743	-0.164

Table 3: Estimated observation matrix $\hat{H} \in \mathbb{R}^{13\times 3}$ after EM convergence. Rows correspond to the 13 elements of the observation vector y_t, and entries represent the loading of each standardized observation on the latent states.

The estimated loading structure is broadly consistent with the intended interpretation of the three latent states. Momentum (M_t) receives the strongest positive loadings from recent runs and boundaries across all window lengths, while wickets load negatively, reflecting the disruptive effect of dismissals on short-term scoring patterns.

Pressure (P_t) is driven primarily by dot balls and wickets, both of which exhibit large positive loadings. Runs load negatively on pressure, indicating that scoring relieves the burden of the chase. These relationships are consistent with the interpretation of pressure as capturing the mathematical and strategic difficulty faced by the batting side.

Batting strength (S_t) exhibits a more diffuse loading pattern. Longer run windows load positively, whereas dot balls load negatively, suggesting that strength captures persistent scoring ability rather than short-term fluctuations. Compared with momentum and pressure, the loading structure for batting strength is less concentrated, indicating that it reflects a broader combination of batting quality and innings stability.

4.1.4 Process Noise Covariance Q

The process noise covariance matrix ($Q \in \mathbb{R}^{3\times3}$) governs the degree to which the latent states are allowed to evolve unpredictably between consecutive deliveries. To reflect the differing temporal characteristics of the three states, (Q) is specified as a diagonal matrix:

$$Q = \begin{bmatrix} 0.05 & 0 & 0 \\ 0 & 0.25 & 0 \\ 0 & 0 & 0.10 \end{bmatrix}.$$

The variances correspond to batting strength (S_t), momentum (M_t), and pressure (P_t), respectively. Momentum is assigned the largest variance to permit rapid adaptation to short-term scoring fluctuations, while batting strength is assumed to evolve more gradually. Pressure is assigned an intermediate variance, reflecting its tendency to build and dissipate over several deliveries.

The matrix (Q) is fixed rather than estimated during the EM procedure. This reduces potential identifiability issues between process and observation noise and improves the stability and interpretability of the estimated matrices ($\hat{H}$) and ($\hat{R}$).

4.1.5 Observation Noise Covariance R

The observation noise covariance matrix $R \in \mathbb{R}^{13\times13}$ quantifies the reliability of the observable features in y_t as signals of the latent states. Unlike the process noise matrix Q, which is fixed, $\hat{R}$ is estimated from the data using the EM procedure. The matrix is initialized as I_{13} and updated according to

$$R_{\text{new}} = \text{diag}\left(\frac{1}{T}\sum_t (y_t - H\hat{x}_{t|t})(y_t - H\hat{x}_{t|t})^\top\right),$$

using a damping rate of 0.25 and a minimum variance floor of 10^{-4} to prevent numerical collapse. For numerical stability, $\hat{R}$ is constrained to remain diagonal, implying independent observation noise across features.

The final estimated diagonal elements are reported in Table 4.

Observation	Variance
Runs (6 balls)	0.3856
Runs (12 balls)	0.0006
Runs (24 balls)	0.3233
Wickets (6 balls)	0.4684
Wickets (12 balls)	0.0001
Wickets (24 balls)	0.4325
Dot balls (6 balls)	0.3984
Dot balls (12 balls)	0.0005
Dot balls (24 balls)	0.3972
Boundaries (6 balls)	0.5592
Boundaries (12 balls)	0.2002
Boundaries (24 balls)	0.4178
Current run rate	0.7090

Table 4: Diagonal elements of the estimated observation noise covariance matrix $\hat{R}$. Off-diagonal elements are constrained to zero. Larger values correspond to noisier observations and therefore lower weight in the filtering step.

Several patterns are noteworthy. Current run rate exhibits the largest observation variance, suggesting that instantaneous scoring rates are relatively noisy indicators of the underlying match state. In contrast, rolling-window aggregates generally display lower variances, indicating that the filter places greater trust in information accumulated over multiple deliveries. Overall, the estimated $\hat{R}$ matrix supports the use of rolling-window statistics as stable signals for latent-state inference.

4.1.6 Numerical Stability Safeguards

Several safeguards were implemented to ensure stable filtering under the highly volatile scoring patterns characteristic of T20 cricket. First, covariance updates employ the Joseph form (Bierman, 2006; Grewal and Andrews, 2014), which preserves positive semi-definiteness under finite-precision arithmetic. Second, the innovation covariance matrix is regularized with a small ridge term ($\epsilon = 10^{-6}$) prior to inversion, with the Moore–Penrose pseudo-inverse used as a fallback for near-singular cases. Third, observation and control features are cleaned before filtering to remove any `NaN` or infinite values arising from edge cases. Finally, filtered states and covariance matrices are monitored for numerical validity throughout the recursive estimation procedure.

Together, these safeguards ensure that the filtering algorithm remains stable even in

the presence of extreme scoring events and highly variable match situations.

4.1.7 Expectation–Maximization Estimation

The matrices $\hat{A}$, $\hat{B}$, $\hat{H}$, and $\hat{R}$ are estimated using an Expectation–Maximization (EM) procedure (Dempster et al., 1977) applied to the training set (IPL 2008–2020; 193,245 deliveries). The process noise covariance matrix Q is fixed throughout estimation, as discussed in Section 4.

In the E-step, the Kalman filter is applied to the training data to compute the sufficient statistics required for parameter estimation. In the M-step, the parameter matrices are updated using regularized least-squares estimators with Tikhonov regularization ($\lambda = 0.05$). Parameter updates are damped between iterations to improve numerical stability and preserve the domain-informed structure of the model.

The algorithm is run for a maximum of 150 iterations and terminates early when the relative change in log-likelihood falls below 10^{-6}.

4.1.8 Win Probability Calibration

Filtered state estimates are converted to win probabilities using the L2-regularized logistic regression model described in Section 2.5. Logistic regression remains one of the most widely used probabilistic calibration methods because it provides well-calibrated probabilities while maintaining interpretability (Niculescu-Mizil and Caruana, 2005; Hosmer Jr et al., 2013). The calibration model is trained exclusively on second-innings deliveries using the 10-dimensional feature vector z_t. The fitted model is subsequently applied without retraining to the validation and test sets.

4.2 Benchmark Models

The proposed Kalman filter framework is benchmarked against three commonly used classification models: logistic regression, random forest, and gradient boosting. Similar comparative evaluations of statistical and machine-learning models have recently been reported in other sports prediction settings (Gifford and Bayrak, 2023).All models are

trained on second-innings deliveries from the training set (IPL 2008–2020) and evaluated on the held-out test set (IPL 2023–2025), with temporal sample weights applied during training.

The benchmark models use the 23 engineered match-state features described in Section 3.2. Hyperparameters are selected using season-blocked cross-validation on the training set. Logistic regression (Hosmer Jr et al., 2013) serves as a linear baseline, while random forest (Breiman, 2001) and gradient boosting (Friedman, 2001) represent nonlinear tree-based ensemble methods.

The proposed Kalman filter framework combines the filtered latent states $(\hat{S}_t, \hat{M}_t, \hat{P}_t)$ with observable match-context variables and interaction terms through the second-stage logistic regression model described in Section 2.5, yielding a 10-dimensional calibration feature vector. Additional hybrid and nonlinear state-space variants were explored as robustness checks but are omitted for brevity, as they did not materially affect the relative ordering of models.

5 Results

We evaluate the proposed Kalman filter framework against logistic regression, random forest, and gradient boosting on the held-out test set (IPL 2023–2025). All model development and hyperparameter tuning are restricted to earlier seasons, preserving temporal ordering and providing a realistic assessment of out-of-sample predictive performance. Performance is evaluated using Brier score (Glenn et al., 1950; Murphy, 1973), log-loss, and area under the ROC curve (AUC) (Fawcett, 2006). Statistical significance is assessed using bootstrap confidence intervals and pairwise Diebold–Mariano tests.

5.1 Overall Predictive Performance

Table 5 reports predictive performance on the held-out test set. Evaluation is performed using second-innings deliveries only, where the eventual match outcome is unambiguously defined at the ball level.

Model	Brier	Log-Loss	ROC-AUC
Gradient Boosting	**0.1844**	**0.5425**	**0.7952**
Random Forest	0.1862	0.5474	0.7905
Kalman Filter	0.1914	0.5582	0.7783
Logistic Regression	0.1927	0.5612	0.7753

Table 5: Predictive performance on the held-out test set (IPL 2023–2025). Lower Brier score and log-loss indicate better performance, whereas higher AUC is preferred.

Gradient boosting achieves the strongest performance across all three evaluation metrics, followed closely by random forest. The superior performance of the tree-based ensembles suggests that nonlinear interactions among match-state variables contain predictive information that is not fully captured by linear models.

The proposed Kalman filter consistently outperforms logistic regression, indicating that the latent-state representation provides predictive information beyond that contained in the observable match context alone. Although the improvement is modest, it is observed across all evaluation metrics and is later shown to be statistically significant.

Overall differences between models are relatively small, suggesting that much of the predictive signal is contained in the observable match state. Consequently, large improvements in aggregate accuracy are difficult to achieve. Nevertheless, the results indicate that the state-space framework provides competitive predictive performance while offering the additional benefit of interpretable latent representations of batting strength, momentum, and match pressure.

5.2 Statistical Significance

To assess uncertainty around the aggregate metrics, Table 6 reports bootstrap 95% confidence intervals following the non-parametric bootstrap procedure proposed by (Efron and Tibshirani, 1994). The intervals overlap substantially across models, reflecting sampling uncertainty at the match-block level, but the point estimates in Table 5 show a consistent ordering across metrics.

Model	Brier 95% CI	AUC 95% CI
Gradient Boosting	(0.1667, 0.2043)	(0.7501, 0.8385)
Random Forest	(0.1693, 0.2005)	(0.7570, 0.8325)
Kalman Filter	(0.1781, 0.2050)	(0.7411, 0.8129)
Logistic Regression	(0.1787, 0.2071)	(0.7326, 0.8138)

Table 6: Bootstrap 95% confidence intervals for Brier score and ROC-AUC on the held-out test set. Point estimates are reported in Table 5.

To formally compare models, we conduct pairwise Diebold–Mariano (DM) tests (Diebold and Mariano, 1995) using ball-level squared prediction errors. Table 7 reports the resulting test statistics and p-values.

Model A	Model B	DM Stat.	p-value
GB	RF	-4.031	< 0.001
GB	KF	-5.510	< 0.001
GB	LR	-6.361	< 0.001
RF	KF	-4.102	< 0.001
RF	LR	-5.020	< 0.001
KF	LR	-5.062	< 0.001

Table 7: Pairwise Diebold–Mariano tests based on ball-level squared prediction errors. GB = Gradient Boosting, RF = Random Forest, KF = Kalman Filter, LR = Logistic Regression.

All pairwise comparisons are statistically significant at the 0.1% level. Of particular interest is the comparison between the Kalman filter and Logistic Regression. Although the improvement in aggregate accuracy is modest, the difference is statistically significant ($p < 0.001$), providing evidence that the latent-state representation captures predictive information beyond that contained in the observable match-state variables alone.

The tree-based ensemble methods significantly outperform the Kalman filter, indicating that nonlinear feature interactions remain an important source of predictive accuracy. Nevertheless, the state-space framework achieves competitive performance while offering interpretable latent representations of batting strength, momentum, and match pressure, a property explored in subsequent sections.

5.3 Calibration Analysis

Calibration assesses whether predicted probabilities correspond to observed outcome frequencies and is widely regarded as a fundamental property of probabilistic prediction

models (Niculescu-Mizil and Caruana, 2005; Guo et al., 2017). Whereas metrics such as ROC-AUC evaluate discriminative ability, calibration measures the accuracy of the probabilities themselves. A perfectly calibrated model predicting a 70% chance of victory should win approximately 70% of the time when issuing such predictions.

Reliability diagrams for the four models are presented in Figure 1. Predicted probabilities are grouped into ten equal-width bins and compared with the corresponding observed win frequencies, following (Bröcker and Smith, 2007). Overall, all four models exhibit good calibration, with most probability bins lying close to the 45-degree line.

The tree-based ensemble methods display minor deviations from perfect calibration at extreme probability values, exhibiting a mild S-shaped pattern commonly observed in flexible nonlinear models. In contrast, the Kalman filter tracks the calibration diagonal closely across much of the probability range, particularly in the intermediate region where many competitive match situations occur.

Quantitative calibration metrics further support this visual assessment. The Kalman filter achieves an Expected Calibration Error (ECE) (Naeini et al., 2015) of 0.0294 and a Maximum Calibration Error (MCE) of 0.0555, implying that predicted probabilities differ from observed outcome frequencies by approximately 2.9% on average, with the largest deviation across probability bins remaining below 5.6%.

Taken together, these results suggest that differences in overall model performance are driven primarily by discriminative accuracy rather than calibration. Although Gradient Boosting achieves the strongest predictive performance, the Kalman filter produces similarly well-calibrated probabilities while retaining the interpretability advantages of the state-space framework.

5.4 Performance by Innings Phase

Tables 8 and 9 report predictive performance by innings and match phase. Performance varies substantially across phases, reflecting the changing information environment of a T20 match. Predictive accuracy improves markedly in the second innings, where the target score provides additional structure to the prediction problem.

Innings	Phase	GB	RF	KF	LR
1st	PP	**0.2380**	0.2402	0.2418	0.2427
1st	Middle	**0.2166**	0.2191	0.2277	0.2305
1st	Death	0.2078	**0.2073**	0.2189	0.2263
2nd	PP	**0.1848**	0.1863	0.1902	0.1889
2nd	Middle	0.1410	0.1433	0.1412	**0.1405**
2nd	Death	**0.0752**	0.0759	0.0933	0.0941

Table 8: Brier scores by innings phase. Lower values indicate better performance; bold values identify the best model within each phase.PP = Powerplay

Innings	Phase	GB	RF	KF	LR
1st	PP	**0.6233**	0.6090	0.6175	0.6175
1st	Middle	**0.7161**	0.7063	0.6807	0.6700
1st	Death	**0.7542**	0.7495	0.7020	0.6799
2nd	PP	**0.8072**	0.8015	0.7827	0.7863
2nd	Middle	0.8808	0.8757	0.8808	**0.8823**
2nd	Death	**0.9600**	0.9597	0.9426	0.9421

Table 9: ROC-AUC by innings phase. Higher values indicate better performance; bold values identify the best model within each phase.

The tree-based ensemble methods deliver the strongest overall performance. Their advantage is most pronounced in the second-innings death overs, where nonlinear interactions among runs required, balls remaining, and wickets in hand become increasingly important. In this phase, both Gradient Boosting and Random Forest achieve Brier scores below 0.08 and ROC-AUC values above 0.95.

The Kalman filter consistently outperforms Logistic Regression throughout the first innings, particularly during the middle and death overs. This suggests that the latent-state representation captures information about batting strength, momentum, and pressure that is not fully reflected in observable match-context variables alone. In the second innings, the gap narrows because the target score provides substantial additional information.

Overall, the phase-level results reinforce the aggregate performance hierarchy. Although tree-based ensemble methods achieve the highest predictive accuracy, the Kalman filter provides a consistent improvement over the static linear benchmark, especially in the less structured first innings.

5.5 Performance by Match Closeness

Table 10 reports Brier scores across match situations. Match closeness is defined using the final winning margin: close matches are decided by 10 runs or fewer or 2 wickets or fewer; moderate matches by 11–30 runs or 3–5 wickets; and one-sided matches by more than 30 runs or more than 5 wickets. The phase rows report performance in the powerplay and death overs across all test matches, regardless of final margin.

Situation	GB	RF	KF	LR	KF Δ
Close	0.2719	**0.2682**	0.2956	0.2951	−0.2%
Moderate	**0.1864**	0.1875	0.2025	0.2041	+0.8%
One-sided	0.1572	0.1611	**0.1529**	0.1546	+1.1%
Death	0.1537	**0.1537**	0.1658	0.1697	+2.3%
PP	**0.2114**	0.2133	0.2162	0.2160	−0.1%

Table 10: Brier scores by match situation on the held-out test set (IPL 2023–2025). Lower values indicate better predictive performance. Bold values denote the best-performing model within each match situation. KF Δ reports the percentage improvement of the Kalman filter relative to Logistic Regression. PP = Powerplay.

Performance varies substantially across match situations. All models perform worst in close matches and best in one-sided contests, reflecting the greater uncertainty associated with tightly contested outcomes.

In close matches, the tree-based ensemble models perform best, while the Kalman filter and Logistic Regression perform nearly identically. This suggests that latent-state tracking provides limited additional information when match outcomes remain highly uncertain throughout. In these settings, nonlinear feature interactions appear more valuable.

The ordering changes in one-sided matches, where the Kalman filter achieves the lowest Brier score among all four models. This finding is consistent with the state-space framework: when a clear match trajectory emerges, the latent states accumulate information about batting strength, momentum, and pressure across deliveries, allowing the model to track growing dominance effectively.

The phase-based rows show that the Kalman filter records its largest improvement over Logistic Regression during the death overs, suggesting that sequential state estimation

becomes more valuable as information accumulates over the innings. In the powerplay, differences between models are small, reflecting the more homogeneous structure of early-innings play.

Taken together, these results indicate that model performance depends strongly on match context. Tree-based ensembles perform best in close, high-uncertainty situations, while the Kalman filter is most competitive when a clear match narrative has developed.

5.6 Feature Importance Analysis

To better understand the drivers of model predictions, Figure 2 reports SHAP-based feature importance rankings (Lundberg and Lee, 2017) for the Hybrid RF+KF model. SHAP values have become one of the most widely adopted model-agnostic explanation methods because they provide consistent feature attributions grounded in cooperative game theory (Molnar, 2020).The complete ranking is provided in Appendix A.2.

Several patterns emerge. First, win probability is driven primarily by chase-context variables. Required run rate, run-rate differential, pressure index, and current run rate are the most influential features, highlighting the central role of the evolving chase situation. Wickets in hand and balls remaining are also highly ranked, confirming that available batting resources are critical determinants of match outcome.

Second, rolling-window features provide additional predictive information. In particular, features aggregated over the previous 24 deliveries tend to rank above their shorter-window counterparts, suggesting that medium-term scoring patterns are more informative than short-lived fluctuations.

Finally, the Kalman filter latent states contribute a smaller but meaningful share of predictive signal. Among the three latent variables, momentum is the most influential, while pressure and batting strength rank somewhat lower. Although their importance is modest relative to the dominant chase-context variables, the latent states rank above many short-window features, indicating that the state-space framework captures information not fully represented by observable match statistics alone.

Overall, the results suggest that prediction is driven primarily by the current state

of the chase, with the latent-state representation providing complementary information beyond conventional match-context variables.

5.7 Kalman Filter Latent State Trajectories

Figure 3 presents average filtered latent-state trajectories for successful and unsuccessful second-innings chases in the held-out test set. The trajectories reveal distinct temporal patterns for batting strength ($\hat{S}_t$), momentum ($\hat{M}_t$), and pressure ($\hat{P}_t$), illustrating how the state-space framework decomposes match evolution into separate underlying components.

Momentum ($\hat{M}_t$) is the earliest differentiator between outcomes. The trajectories diverge within the opening overs and remain separated throughout the innings, with successful chases maintaining consistently higher momentum. This suggests that early scoring patterns contain valuable information about the eventual direction of the chase.

Pressure ($\hat{P}_t$) exhibits the largest and most persistent separation. Successful chases display steadily declining pressure, whereas unsuccessful chases remain under elevated pressure throughout the innings. The sustained gap indicates that pressure provides a compact summary of the evolving difficulty of the chase.

Batting strength ($\hat{S}_t$) follows a different pattern. The trajectories remain relatively similar for much of the innings before diverging sharply during the death overs. Successful chases experience a pronounced late-innings increase in strength, whereas unsuccessful chases stagnate or decline. Batting strength therefore acts primarily as a late confirmation signal rather than an early indicator.

Taken together, the latent trajectories suggest a sequential narrative of successful chases. Momentum separates outcomes early, pressure provides the most persistent distinction throughout the innings, and batting strength emerges as a late-stage confirmation signal. These patterns highlight the interpretability of the state-space framework, providing insight into how and why win probability evolves rather than producing only a single probability estimate.

5.8 Model Limitations and Failure Cases

Despite strong overall performance, all models encounter difficulties in situations characterised by abrupt regime changes or limited historical precedent.

First, unexpected batting collapses remain challenging to predict. Match-state variables may indicate a highly favourable position, yet a rapid sequence of wickets can overturn the outcome within only a few deliveries. Such events are relatively rare and therefore provide limited training signal for statistical learning.

Second, extremely uncommon match situations are inherently difficult to model. Examples include very large required totals in the closing overs or unusual combinations of runs required, balls remaining, and wickets in hand. The scarcity of comparable historical observations limits the reliability of probability estimates in these regions.

Finally, rain-affected matches requiring Duckworth–Lewis–Stern (DLS) adjustments were excluded from the analysis. Because DLS modifies the effective resource structure of the innings, extending the framework to interrupted matches would require a separate modelling approach.

These limitations are shared by all benchmark models considered in this study and reflect the intrinsic uncertainty associated with rare and rapidly evolving match situations.

6 Discussion

6.1 Predictive Performance and the Value of Sequential Modelling

The empirical results reveal a clear performance hierarchy, with Gradient Boosting achieving the strongest overall predictive accuracy, followed by Random Forest, the Kalman filter, and Logistic Regression. This ordering is not unexpected. Tree-based ensemble methods are highly effective at capturing complex nonlinear interactions among match-context variables and therefore possess a natural advantage in situations where the relationships

among runs required, balls remaining, and wickets in hand evolve rapidly.

From the perspective of this study, however, the most informative result is not the aggregate ranking itself, but the consistent improvement of the Kalman filter over Logistic Regression. Although both approaches ultimately map information to win probabilities through a linear calibration layer, the Kalman filter incorporates the temporal evolution of the innings through sequential state updating. The statistically significant improvement over Logistic Regression therefore suggests that information contained in the trajectory of play carries predictive value beyond the current scorecard alone.

This interpretation is reinforced by the latent-state trajectories. Momentum separates successful and unsuccessful chases early in the innings, pressure exhibits the largest and most persistent divergence, and batting strength emerges primarily as a late-stage confirmation signal. Together, these patterns indicate that match evolution contains a structured temporal component that is not fully captured by contemporaneous match-state variables.

The advantage of sequential modelling is particularly evident in one-sided matches, where the Kalman filter achieves the strongest performance among all four models. When a clear match narrative develops, the latent-state representation appears to accumulate information about dominance more effectively than models based solely on current feature values. Conversely, in close matches, where outcomes remain highly uncertain until the final stages, the benefits of latent-state tracking are substantially reduced and flexible nonlinear models provide the greatest advantage.

Taken together, these findings suggest that sequential modelling provides a useful complement to conventional machine-learning approaches in cricket forecasting. Although the Kalman filter does not outperform modern tree-based ensembles in overall predictive accuracy, it demonstrates that the evolution of the innings itself contains meaningful information that can be exploited through a state-space framework.

6.2 Interpretable Match Dynamics

A central contribution of this study is the representation of match evolution through three latent states: batting strength ($\hat{S}_t$), momentum ($\hat{M}_t$), and pressure ($\hat{P}_t$). Rather than producing only a win probability estimate, the model provides a decomposition of the underlying factors driving that probability.

The trajectory analysis suggests that these states capture distinct dimensions of the chase. Momentum acts as an early indicator of eventual success, pressure provides the most persistent distinction between winning and losing chases, and batting strength emerges primarily as a late-stage confirmation signal. Together, these patterns imply that successful chases follow a structured temporal progression rather than arising solely from the contemporaneous match state.

This representation offers a form of interpretability that differs fundamentally from post-hoc explanation techniques (Rudin, 2019). Feature-attribution methods such as SHAP identify which variables contribute to a prediction, but they do not provide an explicit description of how match conditions evolve over time. By contrast, the state-space framework produces latent trajectories that can be interpreted directly as components of match dynamics, yielding a coherent narrative of the innings rather than a collection of feature contributions.

The feature importance analysis further supports this interpretation. Although chase-context variables remain the dominant drivers of prediction, the latent states contribute information beyond the observable feature set, with momentum emerging as one of the most influential derived variables. This suggests that the state-space model captures aspects of match evolution that are not fully represented by conventional rolling-window statistics.

From a practical perspective, these latent states provide information that may be useful to analysts, broadcasters, and teams. In addition to estimating the probability of victory, the framework identifies whether changes in win probability are driven primarily by shifts in momentum, mounting pressure, or underlying batting strength, thereby providing a richer description of match evolution than conventional prediction models

alone.

6.3 Implications for Sports Analytics

The findings highlight an important trade-off between predictive accuracy and interpretability. Tree-based ensemble methods achieve the highest forecasting performance and may therefore be preferred when prediction is the sole objective. The Kalman filter, however, provides a different form of value. It delivers competitive predictive performance while simultaneously producing interpretable latent measures that evolve throughout the innings.

This distinction is particularly relevant in applications where transparency is important. Recent work in sports analytics has likewise emphasized the importance of interpretable predictive models that support coaching and tactical decision-making rather than simply maximizing predictive accuracy (Odet et al., 2025). Broadcast graphics, coaching tools, and decision-support systems often require explanations alongside predictions. In such settings, information about rising pressure or declining momentum may be more useful than a marginally more accurate probability estimate generated by a complex black-box model.

More broadly, the results illustrate the potential of state-space models as a bridge between statistical learning and interpretable sports analytics. Rather than treating prediction and explanation as competing objectives, the framework demonstrates that it is possible to obtain meaningful latent representations while maintaining competitive forecasting performance. Similar approaches may prove useful in other sequential sports settings where outcomes evolve continuously over time.

The framework is also well suited to real-time deployment. Because state estimates are updated recursively after each delivery, predictions can be produced immediately without retraining the model. As sports analytics increasingly moves toward live decision support, methods that combine forecasting with interpretable state tracking are likely to become increasingly valuable.

7 Limitations and Future Work

The proposed framework models match dynamics using only information observable from the ball-by-ball scorecard. While this approach enables broad applicability across historical datasets, it necessarily omits several factors that influence the outcome of a T20 match.

The most important limitation is the absence of player-level information. Batting and bowling resources are represented implicitly through observed outcomes rather than explicitly through player identities and abilities. Consequently, the model reacts to changes in match dynamics only after they are reflected in scoring patterns. Incorporating batter- and bowler-specific information would allow the framework to anticipate changes in match state and provide a more granular description of team resources.

A second limitation arises from the linear-Gaussian assumptions underlying the standard Kalman filter. Although the use of rolling-window aggregates substantially mitigates departures from normality, individual deliveries remain discrete and highly variable events. More flexible state-space formulations, including Unscented Kalman filters, particle filters, and Bayesian state-space models, may provide a richer representation of nonlinear match dynamics while retaining the benefits of sequential estimation.

The analysis is also restricted to uninterrupted IPL matches. Rain-affected matches requiring Duckworth–Lewis–Stern adjustments were excluded because interruptions alter the resource structure of the innings. Extending the framework to accommodate DLS scenarios represents a natural direction for future work. Similarly, evaluating the model on other domestic leagues and international cricket would provide evidence regarding its generalisability across different playing environments.

Beyond win prediction, the latent states themselves may serve as useful analytical tools. Momentum and pressure trajectories could be used to identify turning points in matches, evaluate team strategies, or study performance under varying match conditions. More broadly, the framework illustrates how state-space methods can provide interpretable representations of sporting contests while maintaining competitive predictive performance. As sports analytics increasingly moves toward real-time decision sup-

port, combining forecasting with dynamic state tracking represents a promising avenue for future research.

8 Conclusion

This paper develops a state-space framework for real-time win probability estimation in Twenty20 cricket based on three latent match dynamics: batting strength, momentum, and pressure. Estimated sequentially via Kalman filtering and mapped to win probabilities through a logistic calibration layer, the framework combines competitive predictive performance with an interpretable representation of match evolution.

Empirical results using ball-by-ball data from 1,161 IPL matches show that the proposed approach consistently outperforms a static linear benchmark and remains competitive with modern machine-learning methods. Although tree-based ensemble models achieve the highest overall predictive accuracy, the Kalman filter provides additional insight into how matches unfold through the evolution of its latent states.

The results demonstrate that information contained in the trajectory of play carries predictive value beyond the contemporaneous match state alone. More broadly, the study illustrates how state-space methods can bridge the gap between forecasting and interpretability in sports analytics. As real-time decision support becomes increasingly important, frameworks that combine accurate prediction with dynamic state tracking may offer a valuable complement to purely black-box approaches.

References

Albert, Jim (2015). "Player evaluation using win probabilities in sports competitions". *Wiley Interdisciplinary Reviews: Computational Statistics* 7.5, pp. 316–325.

Asif, Muhammad and Ian G McHale (2016). "In-play forecasting of win probability in One-Day International cricket: A dynamic logistic regression model". *International Journal of Forecasting* 32.1, pp. 34–43.

Baumer, Benjamin S, Gregory J Matthews, and Quang Nguyen (2023). "Big ideas in sports analytics and statistical tools for their investigation". *Wiley Interdisciplinary Reviews: Computational Statistics* 15.6, e1612.

Bierman, Gerald J (2006). *Factorization methods for discrete sequential estimation*. Courier Corporation.

Breiman, Leo (2001). "Random forests". *Machine learning* 45.1, pp. 5–32.

Bröcker, Jochen and Leonard A Smith (2007). "Increasing the reliability of reliability diagrams". *Weather and forecasting* 22.3, pp. 651–661.

Dempster, Arthur P, Nan M Laird, and Donald B Rubin (1977). "Maximum likelihood from incomplete data via the EM algorithm". *Journal of the royal statistical society: series B (methodological)* 39.1, pp. 1–22.

Diebold, Francis X and Roberto S Mariano (1995). "Com paring predictive accu racy". *Journal of Business and Economic Statistics* 13.3, pp. 253–263.

Durbin, James and Siem Jan Koopman (2012). *Time series analysis by state space methods*. Oxford University Press (UK).

Efron, Bradley and Robert J Tibshirani (1994). *An introduction to the bootstrap*. Chapman and Hall/CRC.

Fawcett, Tom (2006). "An introduction to ROC analysis". *Pattern recognition letters* 27.8, pp. 861–874.

Friedman, Jerome H (2001). "Greedy function approximation: a gradient boosting machine". *Annals of statistics*, pp. 1189–1232.

Gama, João et al. (2014). "A survey on concept drift adaptation". *ACM computing surveys (CSUR)* 46.4, pp. 1–37.

Ghahramani, Zoubin and Geoffrey E Hinton (1996). "Parameter estimation for linear dynamical systems".

Gifford, Matt and Tuncay Bayrak (2023). "A predictive analytics model for forecasting outcomes in the National Football League games using decision tree and logistic regression". *Decision Analytics Journal* 8, p. 100296.

Glenn, W Brier et al. (1950). "Verification of forecasts expressed in terms of probability". *Monthly weather review* 78.1, pp. 1–3.

Glickman, Mark E and Hal S Stern (2005). "A state-space model for National Football League scores". *Anthology of statistics in sports*. SIAM, pp. 23–33.

Goel, Rajesh et al. (2021). "Dynamic cricket match outcome prediction". *Journal of Sports Analytics* 7.3, pp. 185–196.

Grewal, Mohinder S and Angus P Andrews (2014). "Kalman Filtering: Theory and Practice with MATLAB 4th Edition". *Wiley-IEEE Press*, p. 640.

Guo, Chuan et al. (2017). "On calibration of modern neural networks". *International conference on machine learning*. PMLR, pp. 1321–1330.

Harvey, Andrew C and Andrew Harvey (1990). *Forecasting, structural time series models and the Kalman filter*. Cambridge university press.

Hosmer Jr, David W, Stanley Lemeshow, and Rodney X Sturdivant (2013). *Applied logistic regression*. John Wiley & Sons.

James, Gareth et al. (2021). "Multiple testing". *An Introduction to Statistical Learning: with Applications in R*. Springer, pp. 553–595.

Kalman, RE (1960). "A New Approach to Linear Filtering and Prediction Problems". *Journal of Basic Engineering* 82.1, pp. 35–45.

Koopman, Siem Jan and Rutger Lit (2015). "A dynamic bivariate Poisson model for analysing and forecasting match results in the English Premier League". *Journal of the Royal Statistical Society Series A: Statistics in Society* 178.1, pp. 167–186.

Lock, Dennis and Dan Nettleton (2014). "Using random forests to estimate win probability before each play of an NFL game". *Journal of Quantitative Analysis in Sports* 10.2, pp. 197–205.

Lundberg, Scott M and Su-In Lee (2017). "A unified approach to interpreting model predictions". *Advances in neural information processing systems* 30.

Molnar, C (2020). "Interpretable machine learning". *Lulu. com*.

Murphy, Allan H (1973). "A new vector partition of the probability score". *Journal of Applied Meteorology and Climatology* 12.4, pp. 595–600.

Naeini, Mahdi Pakdaman, Gregory Cooper, and Milos Hauskrecht (2015). "Obtaining well calibrated probabilities using bayesian binning". *Proceedings of the AAAI conference on artificial intelligence*. Vol. 29. 1.

Niculescu-Mizil, Alexandru and Rich Caruana (2005). "Predicting good probabilities with supervised learning". *Proceedings of the 22nd international conference on Machine learning*, pp. 625–632.

Odet, Arnaud et al. (2025). "A machine learning and explainability-driven methodology for identifying winning strategies in Rugby Union". *Decision Analytics Journal* 15, p. 100568.

Pathak, Neeraj and Hardik Wadhwa (2016). "Applications of modern classification techniques to predict the outcome of ODI cricket". *Procedia Computer Science* 87, pp. 55–60.

Rudin, Cynthia (2019). "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead". *Nature machine intelligence* 1.5, pp. 206–215.

Senevirathne, Hasika KW and Ananda BW Manage (2021). "Predicting the winning percentage of limited-overs cricket using the Pythagorean formula". *Journal of Sports Analytics* 7.3, pp. 169–183.

Shumway, Robert H and David S Stoffer (1982). "An approach to time series smoothing and forecasting using the EM algorithm". *Journal of time series analysis* 3.4, pp. 253–264.

Simon, Dan (2006). *Optimal state estimation: Kalman, H infinity, and nonlinear approaches*. John Wiley & Sons.

Stern, Hal S (2005). "A Brownian motion model for the progress of sports scores". *Anthology of statistics in sports*. SIAM, pp. 257–263.

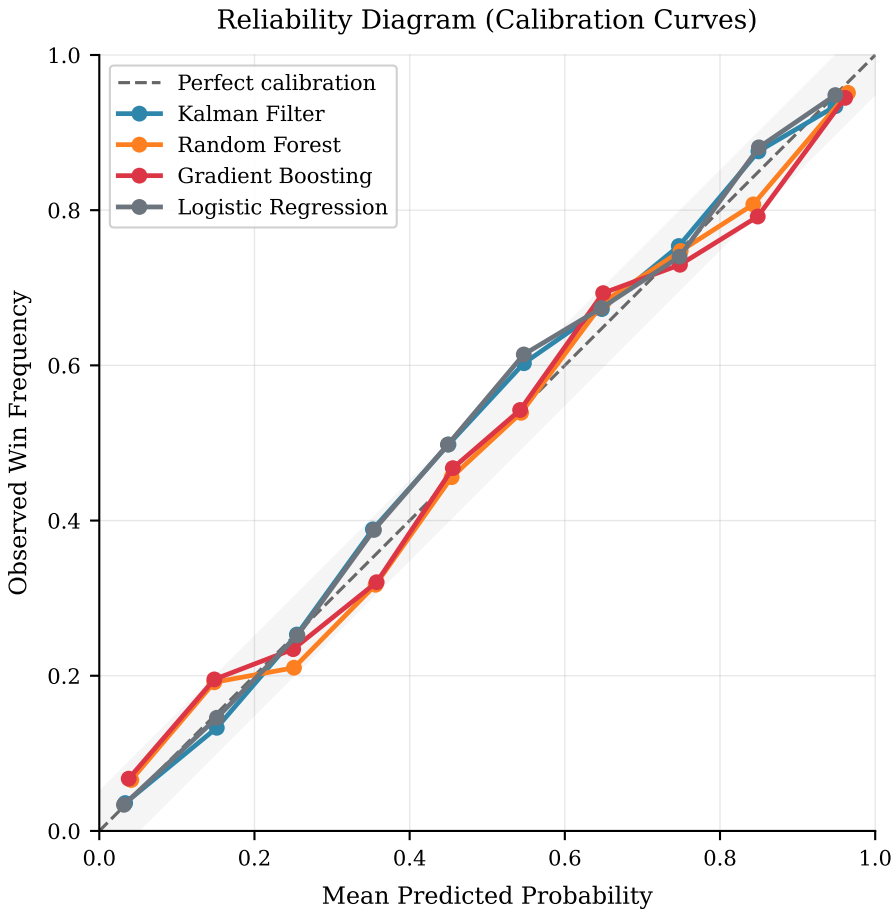
Tango, Tom M, Mitchel G Lichtman, and Andrew E Dolphin (2007). *The book: Playing the percentages in baseball*. Potomac Books, Inc.

Wickramasinghe, Indika (2022). "Applications of Machine Learning in cricket: A systematic review". *Machine Learning with Applications* 10, p. 100435.

Wong, Albert et al. (2025). "A predictive analytics framework for forecasting soccer match outcomes using machine learning models". *Decision Analytics Journal* 14, p. 100537.

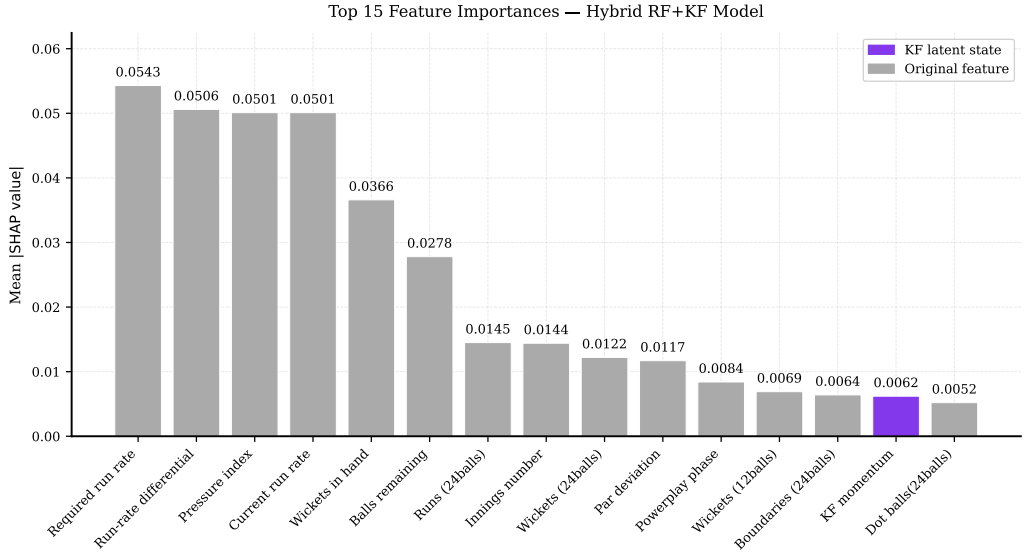
Figures

Figure 1: Calibration Curves



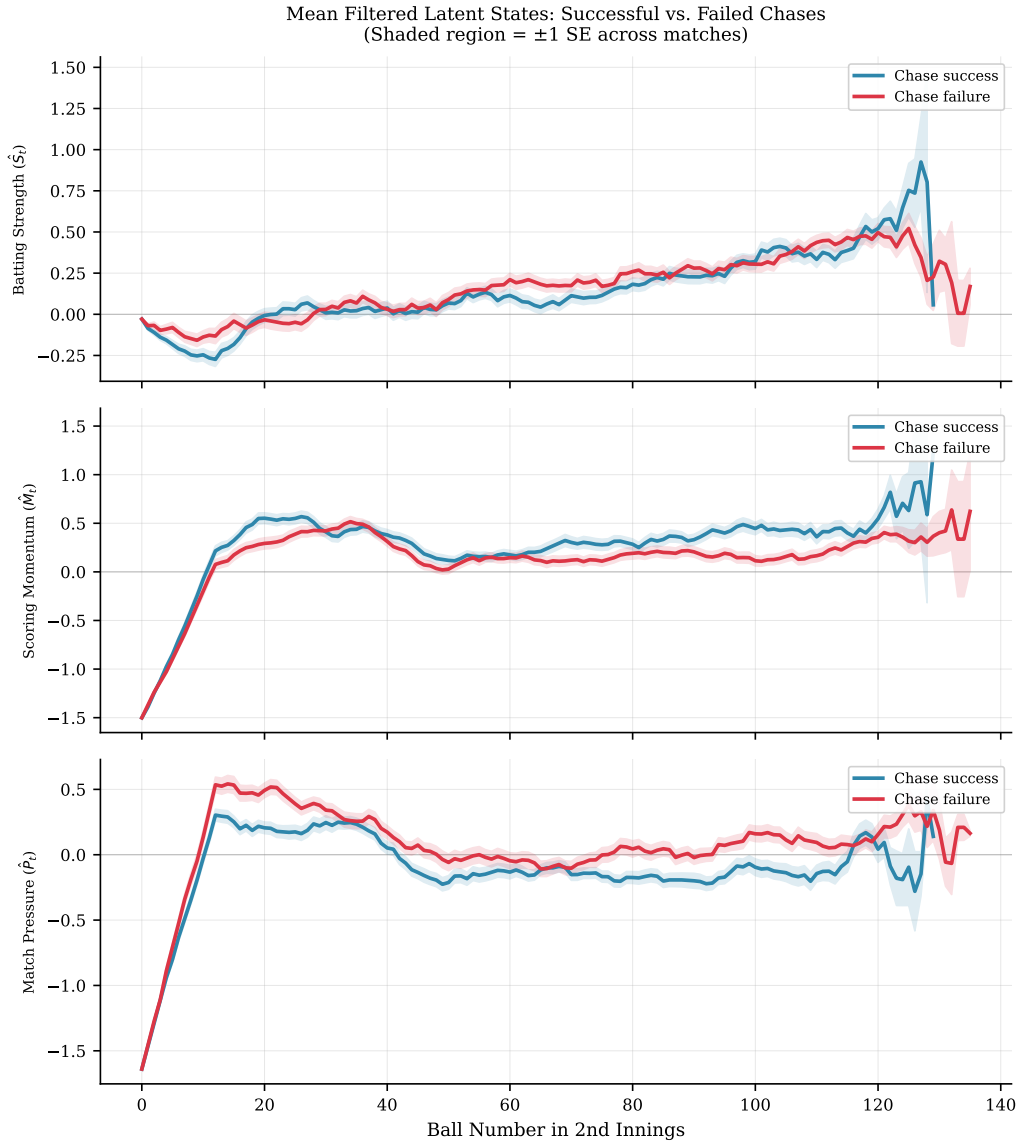
Reliability diagrams for the four models on the held-out test set (IPL 2023–2025). Predicted probabilities are grouped into ten equal-width bins and compared with observed win frequencies. The diagonal line represents perfect calibration, while the shaded region indicates a ±5% tolerance band.

Figure 2: SHAP feature importance rankings - Top 15 Features



Feature importance is measured using mean absolute SHAP values. Purple bars correspond to Kalman filter latent states, while grey bars denote observable match-context and rolling-window features. Required run rate, run-rate differential, and pressure-related variables dominate the rankings, although the latent momentum state remains among the most influential predictors. The complete feature ranking is reported in Appendix A.2

Figure 3: Latent State Trajectories



Average latent-state trajectories for successful and unsuccessful second-innings chases in the held-out test set (IPL 2023–2025). Blue lines denote successful chases and red lines denote unsuccessful chases. Shaded bands indicate ±1 standard error across matches. Momentum ($\hat{M}_t$) separates outcomes early in the innings, pressure ($\hat{P}_t$) exhibits the largest and most persistent gap, and batting strength ($\hat{S}_t$) diverges primarily during the death overs. Together, the trajectories provide an interpretable decomposition of chase dynamics beyond the aggregate win-probability estimate.

A Appendix

A.1 Feature Mapping

Table A1: Feature-to-matrix mapping across model components.

Feature	**Obs.**	**Ctrl.**	**ML**	**Calib.**
Runs (6,12,24)	✓		✓	✓
Wickets (6,12,24)	✓		✓	✓
Dot balls (6,12,24)	✓		✓	✓
Boundaries (6,12,24)	✓		✓	✓
Current run rate	✓		✓	✓
Balls remaining		✓	✓	✓
Wickets in hand		✓	✓	✓
Run-rate differential		✓	✓	✓
Pressure index		✓	✓	✓
Required run rate		✓	✓	✓
Phase indicators		✓	✓	✓
Par deviation			✓	✓
Venue average			✓	✓
Innings number			✓	✓
Latent states ($\hat{S}_t, \hat{M}_t, \hat{P}_t$)				✓
Interaction terms				✓
Total features	**13**	**8**	**23**	**29**

Obs. = Kalman filter observation vector y_t; Ctrl. = control vector u_t; ML = baseline machine-learning features; Calib. = Kalman filter calibration vector z_t.

A.2 Full SHAP Feature Importance Ranking

Table A2 reports the complete SHAP feature importance ranking for the Hybrid RF+KF model. Features are ordered by mean absolute SHAP value, with larger values indicating greater contribution to model predictions. Kalman filter latent states are identified by checkmarks in the KF column.

Feature	**KF**	**Mean** |SHAP|
Required run rate		0.0543
Run-rate differential		0.0506
Pressure index		0.0501
Current run rate		0.0501
Wickets in hand		0.0366
Balls remaining		0.0278
Runs (24 balls)		0.0145
Innings number		0.0144
Wickets (24 balls)		0.0122
Par deviation		0.0117
Powerplay phase		0.0084
Wickets (12 balls)		0.0069
Boundaries (24 balls)		0.0064
KF momentum	✓	0.0062
Dot balls (24 balls)		0.0052
Death phase		0.0049
Runs (12 balls)		0.0044
KF pressure	✓	0.0038
KF strength	✓	0.0037
Middle phase		0.0025
Wickets (6 balls)		0.0019
Boundaries (12 balls)		0.0019
Runs (6 balls)		0.0017
Dot balls (12 balls)		0.0016
Dot balls (6 balls)		0.0009
Boundaries (6 balls)		0.0007
Venue average first innings		0.0000

Table A2: Full SHAP feature importance ranking for the Hybrid RF+KF model. Mean absolute SHAP values are averaged over 2,000 deliveries from the held-out test set (IPL 2023–2025). Kalman filter latent-state features are identified in the KF column. The horizontal rule separates the top 15 features reported in Figure **??** from the remaining features.

A.3 Initialization Design Choices

Several constants used in the initialization procedure were chosen to place the latent states on a scale comparable to that learned during EM estimation. In the first innings, the baseline score of 150 runs and scaling factor of 30 map the typical range of IPL venue averages (approximately 150–210 runs) to moderate values of the batting-strength state, avoiding extreme initialization at high-scoring venues. A default venue average of 175 runs is used when venue-specific information is unavailable.

For second-innings chases, target difficulty is normalized relative to the venue average using a scale factor of 20 runs, corresponding approximately to one additional run per over. Initial batting strength is reduced for above-par targets, while the pressure state increases with target difficulty. Bounds of $[-2, 4]$ are imposed on the initial pressure state to prevent unrealistically large values in unusually high- or low-scoring matches.

Sensitivity analysis indicated that moderate changes to these constants had little effect on overall predictive performance, suggesting that the results are not driven by specific initialization choices.

A.4 Illustrative Match Example

Figure 4: Case Study Plot

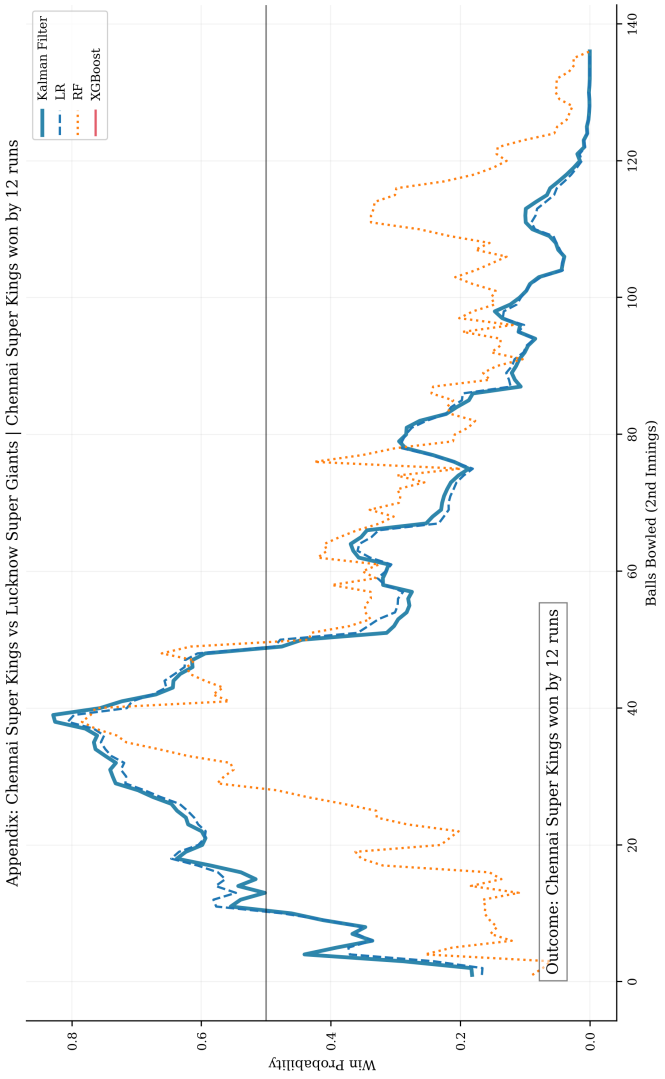


Figure A.1. Ball-by-ball win probability trajectories for a representative Indian Premier League match between Lucknow Super Giants (LSG) and Chennai Super Kings (CSK), in which LSG unsuccessfully chased the target and lost by 12 runs. The figure compares the proposed Kalman filter with the benchmark models. During the early stages of the chase, the Kalman filter assigns LSG a relatively high probability of victory, reflecting favourable latent scoring dynamics and manageable match pressure. Following a sequence of adverse events and a slowdown in scoring, the model reacts more rapidly to the changing match conditions than the benchmark models, producing an earlier and sharper decline in estimated win probability. The example illustrates how sequential state estimation captures the evolving trajectory of the innings, allowing the proposed framework to respond to changing match dynamics before they are fully reflected in the observable scorecard.

A.5 EM Convergence Diagnostics

Figure 5: EM Convergence Plot

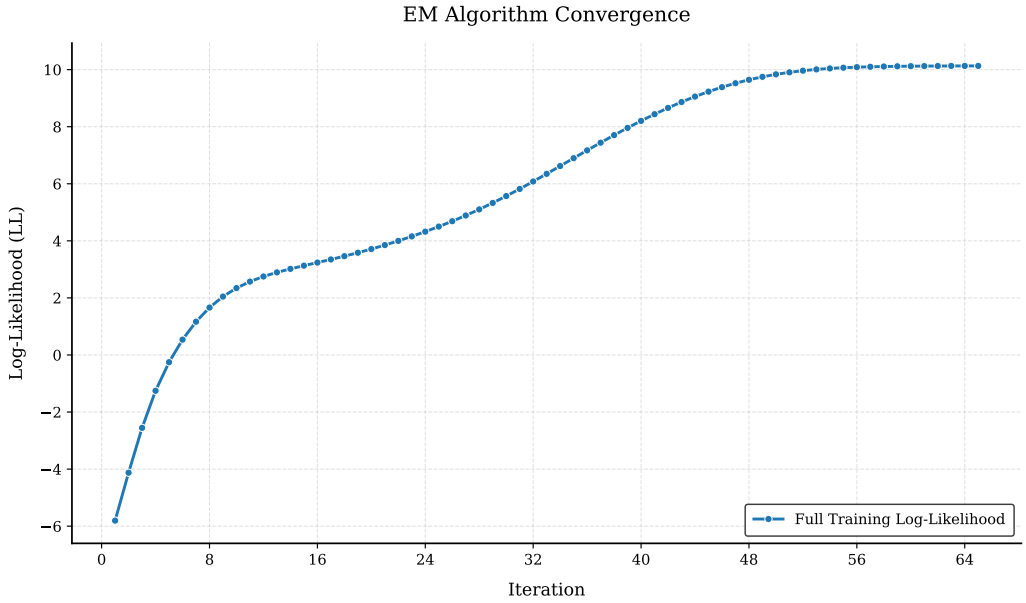


Figure A.2. Evolution of the normalised log-likelihood during Expectation–Maximization (EM) estimation. The objective function increases monotonically and stabilises after approximately 40 iterations, indicating stable convergence of the parameter estimates. Although a maximum of 150 iterations was permitted, the relative improvement became negligible well before the stopping criterion was reached. The absence of oscillatory behaviour suggests that the regularisation and damping procedures described in Section 4 contribute to numerically stable parameter updates.